

Rebuilding the Social Engine

By,
Krithik S

Problem Definition

I rebuild the text-understanding layer with two supervised classifiers: one predicts sentiment, and the other predicts topic. I use only post_text as input. I keep text_id for tracing results and use sentiment_label and topic_category only as targets. I do not use one target as an input to predict the other.

I choose classification because the supplied dataset contains explicit sentiment and topic labels. I do not claim entity recognition or unsupervised topic discovery, since I do not have entity annotations and I already have predefined topic categories.

Data I checked

Check	Result
Original rows	9,000
Missing or blank required values	0
Repeated texts removed	1,100
Conflicting-label text groups	0
Rows retained	7,900
Empty texts after cleaning	0

I remove repeated cleaned texts before splitting, keeping the first occurrence when labels agree. Otherwise, the same wording could appear in both training and testing and overstate performance. I preserve every excluded row and its reason. I stop on duplicate IDs and quarantine conflicting text labels rather than choose an arbitrary label.

Target	Label	Original	Retained
Sentiment	Neutral	3000	2748
Sentiment	Positive	3000	2722
Sentiment	Negative	3000	2430
Topic	Community_Discussion	7752	6803
Topic	Technical_Issues	815	718
Topic	Feature_Feedback	297	250
Topic	Account_Security	136	129

Preprocessing Pipeline

I decode HTML entities, remove HTML-like tags, replace web links with URL and user mentions with USER, then normalize whitespace and lowercase the text. I preserve words such as not and no. I avoid stop-word removal, stemming, and downloaded language models to keep the pipeline small and retain useful sentiment cues.

I convert text to TF-IDF features using single words and adjacent word pairs. I require a term to appear in at least two training documents and cap the vocabulary at 40,000 features. I use sublinear term frequency so repeated words do not grow in weight linearly. I keep the default L2 normalization and smoothed inverse document frequency.

I place cleaning and TF-IDF inside each scikit-learn pipeline. During model selection, I fit the vocabulary and IDF on training text only. After selection, I refit the chosen pipeline on training plus validation data. I never fit those transforms on the held-out test text.

I retain a known preprocessing limitation: 1,092 original rows contain literal Unicode escape sequences. My current cleaner does not decode those sequences. My word tokenizer also drops most standalone punctuation, emojis, and one-character tokens. I would test improvements on a future development split, not retune this reported test result.

Training Methodology

Partition	Rows	Use
Training	5530	Fit each candidate
Validation	1185	Choose by macro F1
Test	1185	Evaluate the selected model
Final fitting	6715	Refit selected model on train + validation

I use a fixed random seed of 42 and a 70/15/15 split of unique cleaned texts. I stratify on the combined sentiment and topic labels so both tasks retain class coverage. I assert that no text ID or identical cleaned text crosses partitions. I save the assignment of every retained row in `split_manifest.csv`.

I compare three fixed candidates per task, choose the highest validation macro F1, and evaluate only the selected learned model on the test set. I also evaluate a majority-class reference baseline. I do not select models from test scores, and I keep the evaluated train-plus-validation models as the submitted artifacts.

Model Selection

I compare Multinomial Naive Bayes, Logistic Regression, and a linear support vector classifier because they work with sparse text features on a CPU. I include a majority-label baseline to check whether learned models add value. I avoid a transformer here because it would add model downloads and heavier dependencies to this small, explainable starting point.

I fix Naive Bayes alpha at 0.5, Logistic Regression C at 2.0, and LinearSVC C at 1.0. I use balanced class weights for the two linear classifiers, which gives smaller classes more weight based on the fitting partition. I allow 2,000 and 10,000 iterations respectively, and stop if convergence warnings appear.

Task	Candidate	Validation accuracy	Validation macro F1
sentiment	MajorityBaseline	34.77%	0.1720
sentiment	MultinomialNB	57.13%	0.5694
sentiment	LogisticRegression	58.14%	0.5812
sentiment	LinearSVC	56.29%	0.5630
topic	MajorityBaseline	86.08%	0.2313
topic	MultinomialNB	86.16%	0.2360
topic	LogisticRegression	91.14%	0.5546
topic	LinearSVC	91.65%	0.5415

For sentiment, I select LogisticRegression with validation macro F1 0.5812. My final fitted vocabulary contains 18,182 features.

For topic, I select LogisticRegression with validation macro F1 0.5546. My final fitted vocabulary contains 18,182 features.

I prioritize macro F1 over accuracy because topic classes are uneven. Macro F1 gives each class equal weight, so a large majority class cannot hide all minority-class mistakes. I use predicted labels only; I do not present an uncalibrated score as a reliable probability.

Evaluation Metrics

I evaluate both selected models on the same 1,185 held-out posts. I report accuracy, macro F1, weighted F1, and balanced accuracy, followed by per-class precision and recall. I treat these as one internal holdout result, not as guaranteed performance on future social posts.

Task	Selected model	Accuracy	Macro F1	Weighted F1	Balanced accuracy
Sentiment	LogisticRegression	58.90%	0.5888	0.5873	59.18%
Topic	LogisticRegression	91.73%	0.5920	0.9059	51.71%

For sentiment, my majority baseline reaches 34.85% accuracy and 0.1723 macro F1. I compare these with the learned model to avoid mistaking class imbalance for useful learning.

For topic, my majority baseline reaches 86.16% accuracy and 0.2314 macro F1. I compare these with the learned model to avoid mistaking class imbalance for useful learning.

I read precision as the share of a predicted class that is correct, and recall as the share of a true class I recover. F1 balances those two. I average class F1 scores equally for macro F1, weight them by class support for weighted F1, and average class recalls for balanced accuracy. I use zero when a precision or F1 denominator is undefined.

Sentiment class results

Class	Precision	Recall	F1	Test support
Negative	0.5950	0.6538	0.6230	364
Neutral	0.5387	0.4891	0.5127	413
Positive	0.6293	0.6324	0.6308	408

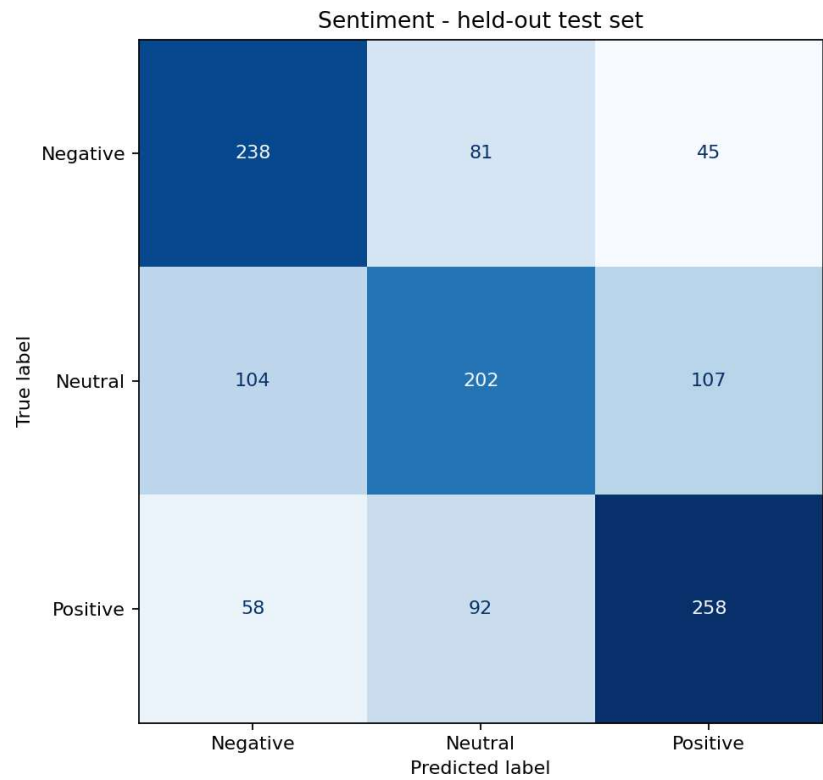
Topic class results

Class	Precision	Recall	F1	Test support
Account_Security	0.8333	0.2632	0.4000	19
Community_Discussion	0.9239	0.9863	0.9541	1021
Feature_Feedback	0.3684	0.1892	0.2500	37
Technical_Issues	0.9714	0.6296	0.7640	108

I see a gap between topic accuracy and balanced performance. In particular, I recover only a small share of Feature_Feedback and Account_Security examples. I would not describe the topic model as equally reliable across categories.

Sentiment Confusion Matrix

I put the supplied labels on rows and my predictions on columns. I read the diagonal as correct predictions and the other cells as errors; each cell contains a count, not a percentage.



I account for all 1,185 test rows in this matrix. I save its numeric values in sentiment_confusion_matrix.csv as well as the chart.

Error Analysis

I found 487 wrong predictions out of the held-out test examples. I keep every error in sentiment_errors.csv so I can inspect more than the examples below.

My largest error direction is Neutral predicted as Positive, with 107 cases. I use this count to identify a weakness, not to change labels or tune on the test set.

TXT_00037 | supplied label: Negative | my prediction: Positive

Text: "I'm going to the duke game on Saturday. Haha. DUKE. What a joke they are."

I read the final phrase about a joke as criticism, while the laughter and game reference provide mixed cues. I would review this as a possible sarcasm or context error. I also notice literal Unicode escape sequences in the source text.

TXT_00086 | supplied label: Positive | my prediction: Negative

Text: "In 7th grade I was introduced to the book Twilight and that's the only book that kept me out of my world."

I read the description of a book keeping the writer out of their world as praise. I suspect the short word-based representation misses that meaning; the prediction alone cannot establish its exact cause.

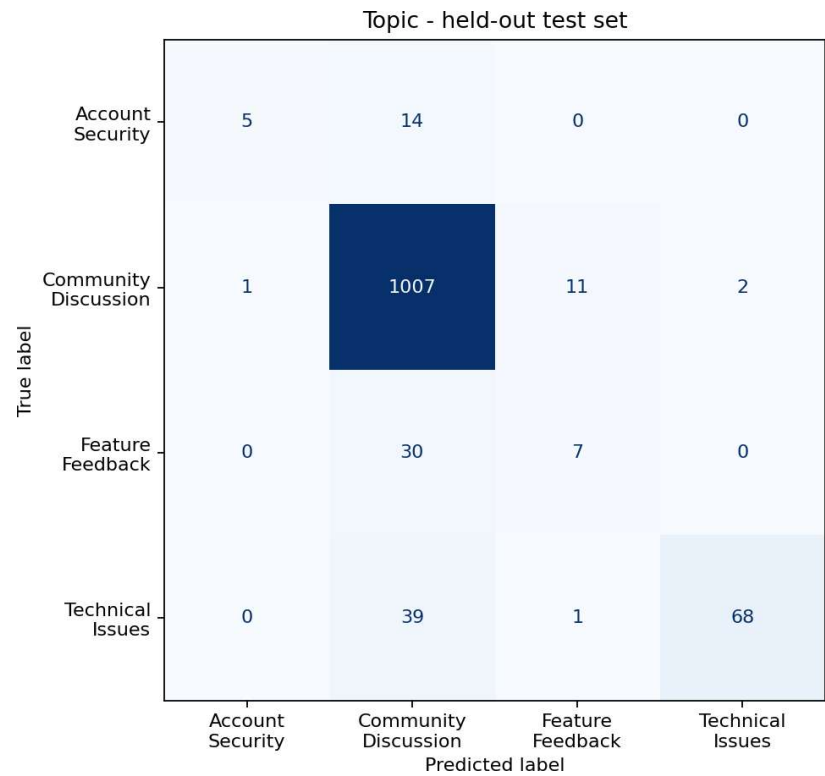
TXT_00108 | supplied label: Negative | my prediction: Neutral

Text: "November 21 in the lonely hour tour.. OKAY SAM SMITH OKAY!!!! :-(TAPOS SOLD OUT PA OKAY!!!! OKAY LANG TALAGA"

I see disappointment about sold-out tickets, an emoticon, and mixed-language text. My word tokenizer does not represent the emoticon well, and I would investigate language coverage.

Topic Confusion Matrix

I put the supplied labels on rows and my predictions on columns. I read the diagonal as correct predictions and the other cells as errors; each cell contains a count, not a percentage.



I account for all 1,185 test rows in this matrix. I save its numeric values in topic_confusion_matrix.csv as well as the chart.

Error Analysis

I found 98 wrong predictions out of the held-out test examples. I keep every error in topic_errors.csv so I can inspect more than the examples below.

My largest error direction is Technical_Issues predicted as Community_Discussion, with 39 cases. I use this count to identify a weakness, not to change labels or tune on the test set.

TXT_00145 | supplied label: Account_Security | my prediction: Community_Discussion Text: "I

know y'all band wagoning j Cole and Kendrick if you thought Black Friday was fire"

I see discussion of musicians rather than account access or security. I would ask for an annotation review, but I keep the supplied label unchanged for scoring.

TXT_00252 | supplied label: Technical_Issues | my prediction: Community_Discussion

Text: ""Whatever happens, I'm going to watch Paper Towns tomorrow. I can't miss this movie.""

I see a plan to watch a movie, with no clear technical failure in the visible sentence. I would review the category definition before treating this as a straightforward model failure.

TXT_00190 | supplied label: Feature_Feedback | my prediction: Community_Discussion

Text: "What's the requirement to work at Dunkin? Complete up to 8th grade? Reasons I hate going there. No people skills."

I see criticism of a workplace or service. I cannot tell whether Feature_Feedback is intended to include this kind of complaint, so I would clarify the labeling policy.

What I can and cannot conclude

I have a reproducible CPU-based classifier for each supplied target, but I do not treat this as a production-ready moderation system. My sentiment result leaves substantial room for improvement. My topic accuracy is dominated by `Community_Discussion`, and the smallest classes have little test support.

I notice possible ambiguity or noise in several supplied topic labels. I also find surface associations in the learned weights, such as `band` for `Account_Security` and `down` for `Technical_Issues`. I treat these as reasons to audit the annotation policy, not proof of how the dataset was labeled. I keep the supplied labels unchanged and score against them.

I check exact normalized duplicates, but I do not establish that near-duplicates, authors, or time periods are independent. I do not have source provenance, annotator agreement, user IDs, or timestamps for a stronger external split. I do not claim causal conclusions, calibrated confidence, multilingual coverage, or a confidence interval from this single split.

My next steps would be to review the category definitions, collect reliable examples for rare topics, and compare character features or contextual models on a new validation design. I would then evaluate once on a fresh holdout. I would use human review for uncertain or consequential predictions.

Deliverables and rulebook coverage

Rulebook requirement	My deliverable	How I run it
Page 2: NLP Model Script/Notebook	run_all.py and src/	python run_all.py
Page 2: Trained Model Files (optional)	models/sentiment_model.pkl and topic_model.pkl	python predict.py --text "I love this update"
Page 2: Evaluation Metrics Report	outputs/Evaluation_Metrics_Report.pdf and metrics.json	python generate_report.py
Pages 2-3: Technical Report and seven sections	outputs/Round2_Technical_Report.pdf	python generate_report.py
Page 3: Confusion Matrix and Error Analysis	Matrix charts, numeric matrices, all-error CSVs, reviewed examples	python check_outputs.py
Page 4: maintained GitHub repository	Source, README, configuration and reproduction evidence	I still need to publish and maintain my repository.